

Eötvös Loránd Tudományegyetem

Informatikai Kar

Programozáselméleti és Szoftvertechnológiai Tanszé

Nagy nyelvi modellek mint kód olvashatósági elemzők

Témavezető:
Szalontai Balázs
egyetemi tanársegéd

Szerző:
Márton Gergő
programtervező informatikus MSc

Budapest, 2026

EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

DIPLOMAMUNKA TÉMABEJELENTŐ

Hallgató adatai:

Név: Márton Gergő

Neptun kód: ZVP7EJ

Képzési adatok:

Szak: programtervező informatikus, mesterképzés (MA/MSc)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Szalontai Balázs

munkahelyének neve, tanszéke: **ELTE IK, Programozáselmélet és Szoftvertechnológia Tanszék**

munkahelyének címe: **1117, Budapest, Pázmány Péter sétány 1/C.**

beosztás és iskolai végzettsége: **Egyetemi tanársegéd, MSc**

Specializáció : Szoftvertechnológia

A diplomamunka címe: Nagy nyelvi modellek mint kód olvashatósági elemzők

A diplomamunka témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben diplomamunka témájának leírását)

A programkód olvashatósága köztudottan az egyik legfontosabb szoftverfejlesztést befolyásoló tényező. Egy nehezen olvasható kódot nehéz fejleszteni és karbantartani. Ezért elengedhetetlen, hogy a kódot mindig olvasható állapotban tartsuk. Az olvashatóság viszont egy nehezen számszerűsíthető fogalom.

Az olvashatóság mérésére több próbálkozás is történt már. Ilyen például Buse és Weimer 2008-ban publikált megközelítése, amely során különböző szintaktikai jellemzőket számoltak, majd ezeket gépi tanulásos modell bemeneteként használva becsülték meg a kód olvashatóságát. Posnet és munkatársainak 2011-es munkája rámutatott arra, hogy a szintaktikai metrikák gyakran gyengén korrelálnak a kód tényleges olvashatóságával. A legújabb módszerek már mély tanuláson alapuló modelleket is javasoltak. Például Dowdell és Henry 2024-es munkája kódreprezentációk tanításával próbálta közvetlenül megbecsülni az olvashatóságot. Napjainkban a nagy nyelvi modellek térnyerése meghatározó a szoftveriparban. Ezek a modellek képesek a szoftvertechnológiai feladatok értelmezésére és megoldására, ami tökéletes alapot adhat a kód olvashatóságának elemzéséhez. Felmerülhet a kérdés, mennyire használhatóak a különböző nagy nyelvi modellek erre a feladatra.

Kutatómunkám célja annak megállapítása, hogy a nagy nyelvi modellek mennyire alkalmasak programkódok olvashatóságának becslésére. Ehhez különféle modellcsaládokat és méreteket értékelek ki egy humán annotált adathalmazon, különböző promptolási stratégiákat alkalmazva. A kutatás során a modellek teljesítményét összevetem a korábban publikált, olvashatóságot becslő módszerek teljesítményével.

Az eredmények értékelése során több, a kód komplexitásához, fenntarthatóságához és értelmezhetőségéhez kapcsolódó metrikát is bevonok az elemzésbe (például ciklomatikus komplexitás). Ez a megközelítés lehetővé teszi annak feltárását, hogy a nagy nyelvi modellek a kód mely aspektusaira alapozzák leginkább az olvashatósági döntéseiket. Ezzel a kutatómunkát a magyarázható mesterséges intelligencia (XAI) oldaláról is igyekszem megvizsgálni, amely napjaink egy kiemelten fontos kérdésévé vált.

Budapest, 2026. 05. 30.

Tartalomjegyzék

1. Bevezetés **2**

2. Kapcsolódó munkák **4**

2.1. Az olvashatóság és az érthetőség fogalma 5

2.1.1. Az olvashatóság első definíciója 5

2.1.2. Olvashatóság és érthetőség különbsége 5

2.1.3. Érzékelt olvashatóság és tényleges megértés 6

2.2. Humán olvashatósági ítéletek 7

2.2.1. A három adathalmaz eredete 7

2.2.2. Egyesített adathalmaz 8

2.2.3. Az értékelők egyetértése 8

2.3. Korábbi olvashatósági modellek 9

2.3.1. Soronkénti jellemzők használata 9

2.3.2. Egy egyszerűbb modell 9

2.3.3. Vizuális és szöveges jellemzők ötvözése 10

2.3.4. Mit jelentenek ezek a modellek a saját összevetésemre? 11

2.4. Nagy nyelvi modellek és a kód olvashatósága 11

2.4.1. Az olvashatóság hatása az LLM-ekre 12

2.4.2. Az LLM-ek mint olvashatósági bírák 12

Irodalomjegyzék **15**

1. fejezet

Bevezetés

Napjainkban a szoftveriparban dolgozó fejlesztők munkájának jelentős részét a meglévő kód megértése, az új kód írása és a kód karbantartása teszi ki. Emiatt minden projekt alapvető feltételévé vált, hogy a hozzá tartozó kód olvasható legyen. Felmerül viszont a kérdés, hogy: „Mit jelent az, hogy egy kód *olvasható*?" Ez önmagában nem egy mérhető fogalom, ráadásul megítélése nagy mértékben szubjektív. Két szakember ugyanazt a kódrészletet vizsgálva számos szempont alapján értékelheti, véleményeik pedig gyakran jelentősen eltérnek egymástól.

Ennek ellenére számos kutatócsoport próbálkozott már különböző módszerekkel, hogy az olvashatóságot mérhetővé, számszerűsíthetővé tegyék. Ilyen Buse és Weimer 2008-ban publikált megközelítése, amely során különböző szintaktikai jellemzőket számoltak, majd ezeket gépi tanulásos modell bemeneteként használva becsülték meg a kód olvashatóságát [1]. Posnet és munkatársainak 2011-es publikációja pedig rámutatott arra, hogy a szintaktikai metrikák gyakran gyengén korrelálnak a kód tényleges olvashatóságával [2]. A korai olvashatósági modellek gépi tanulási módszerekre épültek, a későbbi kutatások pedig már mélytanulást is alkalmaztak.

A modern szoftveriparban azonban megjelent egy új jelenség, amely jelentősen átalakította az ipari szokásokat: a nagy nyelvi modellek (Large Language Model, továbbiakban LLM) elterjedése. Az LLM-ek képesek számnos, egyedi igényű szoftvertechnológiai feladat értelmezésére és megoldására, ami felveti a kutatásom központi kérdését: „Mennyire képesek az LLM-ek kódrészletek olvashatóságának megítélésére, és mennyire használhatók jól olvashatósági bíráként?"

Ez azonban nem egy egyszerű kérdés, mert az LLM-ek megítélése több tényezőtől függően nagy mértékben változhat. Függhet a modell tulajdonságaitól, például

a méretétől, a kvantálásától (quantization), a modellcsaládtól, illetve attól, hogy a válasz előtt „gondolkozhat”-e (thinking). Függhet továbbá attól is, hogy a kérdés milyen módon lett megfogalmazva, vagy akár hogy a feladat mellé kap-e mintapéldákat (promptolási stratégia). Emellett az LLM-ek alapértelmezett mintavételezés mellett nem determinisztikusak (temperature).

Ezek a tényezők a vizsgálatom „tengelyeit” is kijelölik, hisz ha egy LLM ítélete függ a modell méretétől, kvantálásától, családjától, gondolkodási módjától vagy a prompt formájától, akkor a központi kérdésre csak úgy adható megfelelően kielégítő válasz, ha ezeknek a tényezőknek a hatását egyenként is megvizsgálom. Önmagában az sem elegendő, ha egy modell pontszáma jól egyezik az emberekével. Tudni kell azt is, hogy milyen kódtulajdonságokkal mozog együtt az ítélete, és hogy jobb-e a korábbi, nem LLM-alapú módszereknél. A központi kérdés mellett ezért a következő mellékkérdéseket is felteszem:

- Milyen mértékben korrelálnak a modellek pontszámai a humán pontszámokkal?
- Mely modellek teljesítenek jobban illetve rosszabbul a korreláció szerint?
- Számít-e a modell mérete?
- Van-e különbség ugyanannak a modellnek a különböző kvantált változatai között?
- Milyen hatással van az eredményekre, ha a modell gondolkodhat válaszadás előtt?
- Milyen befolyással bír a különböző promptolási stratégiák alkalmazása?
- Mely kódmetrikákkal mozognak együtt a modellek, illetve a humán értékelők pontszámai?
- Hogyan teljesítenek a modellek a korábban publikált megközelítésekhez képest?

2. fejezet

Kapcsolódó munkák

Ez a fejezet azokat a kutatásokat tekinti át, amelyekre a dolgozat közvetlenül épít. Minden hivatkozott munka esetén azokat a részeket emelem ki, amelyek a saját vizsgálatom szempontjából relevánsak. A fejezet felépítésében az alábbi struktúrát követem:

- Először szó lesz arról, hogy mit jelent az olvashatóság, és miben különbözik az érthetőségtől. Ez egy fontos alapot nyújt, hiszen a dolgozat során mind az emberi értékelések, mind a modellek az érzékelt olvashatóságot pontozzák.
- Ez után bemutatom, hogyan készült az a humán értékelésű adathalmaz, amelyen a kísérleteim futottak, és mennyire értenek egyet egymással az emberi értékelők. Innen ered a fő mérőszám, a Spearman-korreláció választása is.
- Szó fog esni a korábbi, humán pontszámokból tanuló olvashatósági modellekről is. Ezek adják azt a viszonyítási alapot, amelyhez az LLM-eket hasonlítom, és jellemzőik (kódjellemzők avagy kódtulajdonságok) egy része a saját metrikáim között is szerepel.
- Ezt követi a nagy nyelvi modellekről szóló rész, mely először azt mutatja be, hogyan hat a kód olvashatósága az LLM-ek munkájára, majd a dolgozathoz legközelebb álló munkát, a CoReEval benchmarkot tárgyalom. Itt lesz szó arról is, hogy mely kérdéseket hagyta nyitva a benchmark, és ezek közül melyekre keresek választ a saját munkámban.
- Végül az utolsó szekcióban indoklom meg, hogy miért korrelációs módszerrel vizsgálom azt, hogy mivel mozog együtt a modellek ítélete.

Szeretném leszögezni, hogy bár a tárgyalt munkák kapcsolódnak, több esetben szorosan is, az általam vizsgált kérdésekhez, de több olyan kérdést is vizsgálnak, amelyek nem képezik részét a dolgozatnak. Ilyenek például az értékelői szerep (persona) hatásának vagy a modellek indoklásainak elemzése.

2.1. Az olvashatóság és az érthetőség fogalma

2.1.1. Az olvashatóság első definíciója

Ahhoz, hogy az olvashatóságot mérni lehessen, először meg kell határozni annak fogalmát. A természetes nyelvű szövegeknél például ez egy régóta megoldott kérdés. A Flesch-Kincaid-index és társai egyszerű mennyiségekből, például a mondatok és a szavak hosszából becsülik meg, hogy mennyire könnyű egy szöveget elolvasni. Buse és Weimer ilyen mérőszámot szerettek volna megalkotni a forráskódokhoz is, ezért a fogalmat az emberi ítéletből vezették le. Az olvashatóság szerintük **egy emberi ítélet arról, mennyire könnyű megérteni egy szöveget** [1].

2.1.2. Olvashatóság és érthetőség különbsége

Ez a meghatározás azonban szándékosan tág, ezért a szerzők azt is kimondják, hogy mi nem tartozik bele. Brooks megkülönböztetésére építve a komplexitást a szoftver „lényegi” tulajdonságának tekintik, amely a megoldandó problémából adódik. Az olvashatóság ezzel szemben pedig egy „járulékos” tulajdonság, amit a kódot megíró fejlesztő befolyásolhat. Ugyanígy elhatárolják az olvashatóságot a kódolási szabványoktól is, mert azok elsősorban az egységességet szolgálják, nem pedig (közvetlenül) az olvashatóságot [3]. Ezek az elhatárolások így a mérés módját is meghatározták. Rövid, kontextus nélküli részleteket pontoztattak, és olyan jellemzőket választottak, amelyek nem függnek a kód méretétől, hogy a mérés minél kevésbé tükrözze a megoldott feladat bonyolultságát.

Ez azonban egy fontos problémát hordoz magában. Az olvashatóságot ugyanis a megértés könnyűségével írják le, miközben nem a *megértést* mérik, hanem azt, hogy az értékelő mennyire tartja *könnyen érthetőnek* a kódot. Bővített cikkükben Buse és Weimer maguk is az érthetőség „járulékos” összetevőjének nevezik az olvashatóságot [3].

Posnett és munkatársai szerint a szakirodalom gyakran összemossa az olvashatóság, az érthetőség, a karbantarthatóság és a komplexitás fogalmát, ezért élesebb határt húznak. Azzal a párhuzammal éltek, hogy az olvashatóság a szintaktikai, míg az érthetőség a szemantikai elemzésnek felel meg. Az olvashatóság szerintük egy érzékelt akadály, amelyet a programozónak „meg kell ugrania", mielőtt a kód jelentésével foglalkozni tudna [2].

Scalabrino és munkatársai ugyanezt úgy fogalmazták meg, hogy az olvashatóság azt méri, mekkora erőfeszítés kell a kódban lévő információ eléréséhez, az érthetőség pedig azt, mennyire összetett maga ez az információ [4]. A két fogalom szubjektivitása is más forrásból ered szerintük. Az olvashatóság ugyanis a személyes ízléstől és szokásokból függ, míg az érthetőség a fejlesztő előzetes tudásából, például abból, hogy ismeri-e a használt API-kat [5].

A legújabb, LLM-eket vizsgáló munka már kifejezetten így választja szét a kettőt. Az olvashatóság a kód vizuális feldolgozhatósága (elrendezés, behúzás, elnevezés), az érthetőség a szándék, a logika és a viselkedés megértése [6].

2.1.3. Érzékelt olvashatóság és tényleges megértés

A különbségtétel tehát fokozatosan élesedett. Amint felmerült viszont a kérdés, hogy ezek a modellek a tényleges megértést is előre jelzik-e, a két fogalmat külön kellett kezelni. Amíg az olvashatósági modelleket kizárólag azzal érvényesítették, hogy mennyire egyeznek az emberek érzékelt ítéletével, addig erre nem volt szükség. A dolgozat szempontjából ez azért fontos, mert az általam vizsgált LLM-ek is érzékelt olvashatóságot pontoznak, és a humán pontszámok, amelyekhez hasonlítom őket, szintén ezt bírálták.

Azonban ha az olvashatóság és az érthetőség valóban különbözik, akkor egy jó olvashatósági pontszám sem garantálja, hogy a fejlesztő meg is érti a kódot. Ezt a feltevést Scalabrino és munkatársai empirikusan is megvizsgálták [5]. Kísérletükben nem azt kérdezték a résztvevőktől, mennyire olvasható egy kód, hanem azt kérték, hogy értsék meg azt. A résztvevők ezután jelezték, hogy megértették-e a metódust vagy sem. Ha igen, három ellenőrző kérdésre kellett válaszolniuk. A szerzők a válaszok helyességét és a megértéshez szükséges időt is mérték. Ez garantálta, hogy az érzékelt megértés mellett a tényleges megértésről is volt adatuk. A 46 résztvevő tíz nyílt forrású Java-rendszer 50 metódusát vizsgálta. Ezekre a szerzők 121 met-

rikát számoltak ki, köztük a korábbi olvashatósági modellek jellemzőit, egy teljes olvashatósági modellt és a ciklomatikus komplexitást.

Az eredmény negatív lett. Egyik metrika sem mutatott érdemi kapcsolatot a megértéssel. A legerősebb Kendall-korreláció is csak $|\tau| \approx 0{,}18$ volt, ami a szerzők által használt sávok szerint is csupán egy gyenge kapcsolatot jelez. A különbséget a következő példa jól szemlélteti. Egy Weka-metódusnál minden résztvevő azt jelezte, hogy megértette, és az ellenőrző kérdések közül kettőre a háromból helyesen válaszoltak. Az olvashatósági modell viszont ugyanennek a metódusnak az olvashatóságára mindössze 0,14-es valószínűséget adott [5].

A dolgozat szempontjából ez egy érvényességi korlátként tekinthető (*thread to validity*). Ahogy arról már szó volt, a saját vizsgálatomban mind az emberi, mind a modell által adott pontszám érzékelt olvashatóság. Azaz, ha egy modell pontszáma jól korrelál az emberekével, abból nem következik, hogy a modell érti a kódot, vagy hogy a kód valóban érthető.

2.2. Humán olvashatósági ítéletek

Az olvashatósági modellek és a saját vizsgálatom is ugyanarra épül: emberek által adott olvashatósági pontszámokra. Ezek jelentik a „helyes választ”, amelyhez minden automatikus módszert mérnek. Ezért fontos elemezni, hogy kik és milyen körülmények között adták ezeket a pontszámokat, és mennyire értenek egyet egymással.

2.2.1. A három adathalmaz eredete

Az első nagy humán olvashatósági adathalmazt Buse és Weimer gyűjtötte. 120 informatikus hallgató egy ötfokozatú skálán pontozott 100 rövid Java-részletet, így 12 000 ítélet született. Minden kódrészlet pontosan három egymást követő, egyszerű utasításból állt. Közbeeső megjegyzésekkel és üres sorokkal együtt így átlagosan 7,7 sor volt egy kódrészlet. Az olvashatóságot a résztvevőknek szándékosan nem definiálták, hogy a saját, intuitív fogalmukat kövessék [1].

Dorn adathalmaza ennél jóval nagyobb és változatosabb. Ez az adathalmaz 360 részletet tartalmaz Java, Python és CUDA nyelveken, nyelvenként egyenlő arányban 10, 30 és 50 soros részletekkel. A pontszámokat 5468 értékelő adta, köztük 1800

ipari fejlesztő. Fontos megjegyezni viszont, hogy egy értékelő viszont átlagosan csak 14 részletet látott. A pontszámok különbségei így részben az értékelők közötti különbségekből is adódhatnak [4]. A harmadik adathalmazt Scalabrino és munkatársai építették, mert a korábbi részletek gyakran nem teljes kódegységek voltak. Szöveges jellemzőiknek így egy része nem volt kiszámolható ezeken a kódrészleteken. Négy nyílt forrású Java-projekt 10-50 soros metódusaiból 200-at választottak ki, viszont a kiválasztás nem volt véletlenszerű. A metódusokat jellemzőik alapján úgy válogatták ki, hogy egymástól minél jobban különbözzenek. Mind a 200 metódust kilenc hallgató értékelte [7].

2.2.2. Egyesített adathalmaz

Ennek a három forrásnak a Java-nyelvű részhalmazait a `se2p/code-readability-merged` adathalmaz egyesíti. Ez összesen 421 részlet: 100 a Buse-, 121 a Dorn- és 200 a Scalabrino-adathalmazból [6]. Ez az egyesített adathalmas képezi a saját kísérleteim alapját. Az egyesített adat tehát három különböző értékelői körből, eltérő hosszúságú és eltérő módon kiválasztott részletekből áll.

2.2.3. Az értékelők egyetértése

Buse és Weimer bővített cikke az értékelők egyetértését öt különböző statisztikával is megvizsgálja. A szerzők érvelése szerint mivel az értékelők nem kaptak pontos útmutatást, az abszolút pontszámnál fontosabb a relatív sorrend. Eszerint ha két értékelő ugyanabba a sorrendbe állít két részletet, akkor egyetértenek, még ha más számot adtak is. Ezért választják fő mérőszámnak a **Spearman-féle rangkorrelációt**. Egy átlagos értékelő és a csoportátlag Spearman-korrelációja 0,55 volt. A szerzők ezt úgy értelmezik, hogy az emberek jelentősen, de nem túlnyomó mértékben értenek egyet abban, milyen az olvasható kód, és hogy az átlagolás megtartja a közös tényezőket, a személyes ízlésből adódókat pedig kiszűri [3]. A saját vizsgálatom ebből a mérőszámot veszi át. A modellek pontszámait a humán átlagpontszámokhoz hasonlítom, és itt is a sorrend egyezése a lényeges, ezért a fő mérőszám nálam is a Spearman-korreláció. Erre épül a modellek, a modelltulajdonságok és a promptolási stratégiák összehasonlítása is.

2.3. Korábbi olvashatósági modellek

A humán pontszámok összegyűjtése után a következő kérdés az volt, hogy ezek az ítéletek vajon megjósolhatóak-e automatikusan mérhető kódtulajdonságokból. Az erre épülő modellek mind felügyelt tanulással (*supervised learning*) készültek. A humán pontszámokat egy küszöbbel „olvasható" és „nem olvasható" címkékre alakították. Ezeken a címkéken pedig betanítottak egy osztályozó modellt. A fejlődésük három lépésben követhető, és minden lépés az előző modell egy gyengeségére válaszolt. A dolgozat szempontjából ezek a modellek három okból fontosak. Ezekhez hasonlítom az LLM-eket (ez az utolsó mellékkérdés), jellemzőik egy része a saját metrikáim között is szerepel, és jól mutatják, miért nehéz különböző módszerek eredményeit egymás mellé tenni.

2.3.1. Soronkénti jellemzők használata

Buse és Weimer 25 jellemzőt használtak. Mindegyik egy egyszerű mennyiség soronkénti átlaga vagy maximuma volt. Ilyen például a sorhossz, az azonosítók száma és hossza, a behúzás vagy akár a megjegyzések és az üres sorok száma. A kódrészleteket két csoportra osztották a 3,14-es átlagpontszámnál, mert az átlagok eloszlásának két csúcsa között ez volt természetes határ. A legjobb osztályozók a részletek 75-80%-át sorolták helyesen [1]. A bővített cikk az egyik osztályozó folytonos kimenetét is összeveti a humán átlaggal. A Spearman-korreláció 0,71, ami magasabb, mint egy átlagos értékelő esetében (0,55). Meg kell jegyezni azonban, hogy ez az összevetés azonban nem független teszt, mert a modellt éppen arra az átlagra tanították, amellyel összehasonlítják. A jellemzők közül az azonosítók és a karakterek soronkénti száma bizonyult a legerősebbnek, az átlagos azonosítóhossznak viszont szinte semmilyen előrejelző ereje nem volt, és a megjegyzések szerepe is csak mérsékelt lett [3].

2.3.2. Egy egyszerűbb modell

Posnett és munkatársai ugyanezen a 100 kódrészleten két ponton is bírálták a Buse-modellt. Az egyik a modell mérete, a másik pedig a méretfüggetlenség. Az általános szabály szerint egy prediktorra 10-20 megfigyelés kell. Ez 100 részlet esetén legfeljebb 5-10 jellemzőt jelent, nem 25-öt. Buse és Weimer szerint a jellemzőik

nem függnek a részlet hosszától, Posnették viszont kimutatták, hogy ezen jellemzők közül több erősen korrelál a karakter- vagy szószámmal. Helyettük három jellemzőt javasoltak: a sorok számát, a Halstead-féle térfogatot és a bájtszintű entrópiát. Az ezekre épülő modellek a Buse-modellnél szignifikánsan jobban teljesítettek. A cikk az egyes metrikák és a humán átlag Spearman-korrelációját is közli. A Halstead-térfogatra $\rho = -0{,}64$, a tokenentrópiára $\rho = +0{,}62$, a sorok számára viszont csak $\rho = +0{,}23$ adódott [2]. A szerzők értelmezése szerint ez azt jelenti, hogy azonos méret mellett a több és változatosabb token rontja az olvashatóságot, a több sor viszont javítja, mert a tokenek jobban széthúzódnak. Ez az elemzés kiváló alapot ad a dolgozatom metrikaelemzésének. A lényeges különbség, hogy a Posnették egy kisebb adathalmazzal dolgoztak.

2.3.3. Vizuális és szöveges jellemzők ötvözése

A következő lépés azt vette figyelembe, hogy a kódot az ember a képernyőn látja, és szövegként is olvassa. Dorn modellje a kód vizuális megjelenését is leírta. Ilyen például a behúzás szabályossága és az egymás alá igazított műveletek. Ehhez készült a már említett, három nyelvet felölelő adathalmaz. Scalabrino és munkatársai pedig az azonosítók és a megjegyzések fontosságából indultak ki. Ők szöveges jellemzőket javasoltak. Ilyen volt többek között a megjegyzések és az azonosítók szóegyezése (CIC), a szótári szavakból álló azonosítók aránya (ITID), a kódblokkok szóhasználatának koherenciája (TC) vagy a megjegyzések olvashatósága a Flesch-Kincaid-index szerint (CR). Ezek a szöveges és szerkezeti jellemzők kiegészítik egymást, ugyanis a részletek 12-21%-át csak a szöveges jellemzőkre épülő modell sorolta helyesen [7]. A folyóiratváltozat minden addigi jellemzőt egyetlen modellben egyesített. Ez a modell pedig a három adathalmaz 660 kódrészletén átlagosan 84,4%-os pontosságot és 0,867-es AUC-t (*Area Under the ROC Curve*) ért el, valamint az összevont tesztben minden korábbi jellemzőkészletnél szignifikánsan jobb volt. A szöveges jellemzők közül a megjegyzések olvashatósága (CR) bizonyult a legjobb előrejelzőnek [4]. Ezek a jellemzők a saját, LLM-alapú metrikáim közül többhöz is kapcsolódnak, például a CIC a `consistency_of_comments_and_code`, a CR a `clarity_of_comments` metrikával. A lényeges különbség az, hogy Scalabrinóék képlettel számolják őket, én pedig LLM-értékelőkkel mérem.

Összefoglalva tehát a három modell a következőképpen teljesít a humán pont-

számok előrejelzésében:

Munka	Módszer	Adat	Közölt eredmény
Buse és Weimer [1, 3]	25 soronkénti jellemző és Weka-osztályozók alkalmazása	100 Java-részlet 120 hallgató értékelésével	A legjobb osztályozók esetén 75-80% pontosság, valamint $\rho = 0{,}71$-es modellátlag
Posnett és mtsai [2]	3 jellemző alkalmazása	100 kódrészlet (lásd Buse és Weimer)	A Buse-modellnél szignifikánsan jobb ROC, F-mérték és találati arány
Scalabrino és mtsai [4]	Szerkezeti, vizuális valamint szöveges jellemzők	660 kódrészlet	84,4% súlyozott pontosság és 0,867-es AUC

2.1. táblázat. A korábbi olvashatósági modellek összefoglalása.

2.3.4. Mit jelentenek ezek a modellek a saját összevetésemre?

A 2.1. táblázatból javarészt látható, miért nem lehet ezeket az eredményeket egyszerűen a saját számaimmal összevetni. A modelleket adathalmazonként külön tanították és keresztvalidációval értékelték, valamint a pontosság vagy az AUC bináris címkékre vonatkozik. A folytonos, humán pontszámmal való rangkorrelációt modellszinten csak Buse és Weimer bővített cikke közli, az is a tanítóadaton. Az általam vizsgált LLM-ek viszont semmilyen tanítást nem kapnak ezen az adaton, és a kimenetük egy pontszám, nem pedig osztálycímke.

2.4. Nagy nyelvi modellek és a kód olvashatósága

Az LLM-ek és az olvashatóság kapcsolata két irányból is vizsgálható. Az egyik kérdés az, hogy a kód olvashatósága hogyan befolyásolja az LLM-ek munkáját. A másik az, hogy az LLM-ek mennyire tudják megítélni a kód olvashatóságát. Bár a dolgozatomban csupán a második kérdéssel foglalkozom, az első is fontos értéket képvisel. Megmutatja ugyanis, hogy a klasszikus olvashatósági jellemzők hogyan függnek össze az LLM-ek viselkedésével.

2.4.1. Az olvashatóság hatása az LLM-ekre

Szalontai és munkatársai azt vizsgálták, hogy vajon az olvashatóbb kódot jobban meg tudják-e magyarázni az LLM-ek. A HumanEvalExplain benchmark 164 rövid Python-programján hat nyílt modellt futtattak, 1,5 és 70 milliárd paraméter között. A modell először magyarázatot írt a kódról, majd ugyanebből a magyarázatból megpróbálta újra előállítani a kódot, amelyet tesztesetek ellenőriztek. Ha az új kód átment a teszteken, a magyarázatot sikeresnek tekintették. Az eredmények szerint a Posnett-féle olvashatósági pontszám a sikeresen magyarázott programoknál magasabb volt. Ez hat modellből ötnél szignifikánsan látszik, az egy kivétel pedig a benchmarkon is leggyengébben teljesítő modell volt. A siker előrejelzésében a sorhossz, a Halstead-térfogat és az entrópia bizonyultak fontosnak [8]. Ezek a mennyiségek a saját metrikáim között is szerepelnek, viszont az én dolgozatom fordítva vizsgálja ugyanezt a kapcsolatot. Nálam ugyanis nem az LLM a vizsgált alany, akinek a kódot meg kell értenie, hanem ő maga a bíráló.

2.4.2. Az LLM-ek mint olvashatósági bírák

A dolgozatom témájához legközelebb Ouédraogo és munkatársai CoReEval benchmarkja áll, amelyben azt vizsgálják, hogy mennyire hasonlít az LLM-ek olvashatósági ítélete az emberekéhez. A benchmark 656 kódrészletből áll, amelyek a fent bemutatott három adathalmazból származnak, három nyelven (Java, Python, CUDA). A szerzők tíz modellt értékeltek ki, négyet API-on keresztül (például GPT-4o, Claude Sonnet 3) és hatot helyben futtatva (4-7 milliárd paraméter). Négy promptolási stratégiát (zero-shot, few-shot, lépésenkénti gondolkodás és több szimulált szakértő konszenzusa a promptban), két értékelői szerepet (hallgató és senior fejlesztő), valamint kilenc dekódolási beállítást használtak. A modellek egy 0 és 20 közötti pontszámot adtak, valamint egy rövid indoklást ehhez. A pontszámot az összevetéshez a készítők egy 1-től 5-ig terjedő skálára konvertálták át [6].

A cikk eredményei a saját mellékkérdéseim közül többre is előzetes választ adnak:

- **Egyezés az emberekkel:** Java-ban az LLM-ek és az emberek pontszámai között a Spearman-korreláció ebben a kísérletben 0,25, Pythonon és CUDA-n nulla körüli. Ugyanezen a Java adathalmazon a Scalabrino-féle jellemzőalapú modell korrelációja 0,27, tehát az LLM-ek nem voltak jobbak a klasszikus

modellnél. A modellek ráadásul optimistábbak az embereknél: a kódrészletek 65%-át ítélték olvashatónak, az emberek csak 44%-át.

- **Promptolás:** Az összetettebb stratégiák (few-shot és a szakértői konszenzus) magasabb pontszámokat adtak, de az emberektől való átlagos eltérés (MAE) náluk volt a legnagyobb. A legkisebb átlagos eltérés pedig a legegyszerűbb, zero-shot promptnál volt megfigyelhető. Az átlagos eltérés azonban a pontszámok eltolódására is érzékeny, ezért keveset mond arról, mennyire jól rangsorolja a modell a részleteket.

- **Méret:** A nagyobb modellek kisebb átlagos eltéréssel pontoztak [6]. Ezt azonban különböző gyártók modelljeinek összevetéséből állapították meg, és a nagy modellek egy részének mérete nem is ismert, így a méret hatása nem választható el a modellcsaládétól.

A szerzők az LLM-ek indoklásait is elemezték. A leggyakrabban említett szempontok a megjegyzések, az elnevezések, a behúzás és a szerkezet voltak. Ezek együtt az indoklások több mint 82%-ában szerepeltek. Ebből arra következtetnek, hogy az LLM-ek elsősorban felszíni jellemzőkre figyelnek. Ha a promptban kifejezetten fejlesztői szempontokat soroltak fel, a modellek ezeket sokkal gyakrabban említették, az egyezés az emberekkel azonban összességében nem javult. A szerzők maguk is megjegyzik, hogy a gyakoribb említés nem jelent mélyebb érvelést [6].

A nagy hasonlóság ellenére viszont a cikk több olyan kérdést nem vizsgál meg, amelyekre a dolgozatomban választ keresek:

- **Kvantálás és gondolkodási mód:** A kvantálással nem foglalkozik, a „gondolkodást” pedig csak promptszinten vizsgálja. Hibrid modellek gondolkodó és nem gondolkodó módját nem veti össze, pedig ezek azonos súlyokkal futnak, így a különbség csak a gondolkodásból adódhat.

- **Méret:** Ahogy fent láttuk, a modellméretet nem különítik el a modellcsaládtól.

- **Statisztika:** A szerzők egyoldali Mann-Whitney-próbát használnak, és többszörös összehasonlítási korrekcióról nem írnak. Ezen felül modellenkénti korrelációt sem közöltek.

- **Magyarázhatóság:** A cikk azt elemzi, mit mond a modell az indoklásában. Azt azonban nem vizsgálták, hogy a pontszáma milyen mérhető kódtulajdonságokkal mozog együtt.
- **Egyetértés és válaszadás:** Az értékelők egyetértésének kiszámítását említi a cikk, de értéket nem közöl, a modellek válaszadási arányát pedig nem elemzi.

Irodalomjegyzék

[1] Raymond P. L. Buse és Westley R. Weimer. „A Metric for Software Readability". *Proceedings of the 2008 International Symposium on Software Testing and Analysis (ISSTA 2008)*. Seattle, WA, USA, 2008, 121–130. old. DOI: 10.1145/1390630.1390647.

[2] Daryl Posnett, Abram Hindle és Premkumar Devanbu. „A Simpler Model of Software Readability". *Proceedings of the 8th Working Conference on Mining Software Repositories (MSR 2011)*. Waikiki, Honolulu, HI, USA, 2011, 73–82. old.

[3] Raymond P. L. Buse és Westley R. Weimer. „Learning a Metric for Code Readability". *IEEE Transactions on Software Engineering* 36.4 (2010), 546–558. old.

[4] Simone Scalabrino és tsai. „A Comprehensive Model for Code Readability". *Journal of Software: Evolution and Process* 30.6 (2018), e1958.

[5] Simone Scalabrino és tsai. „Automatically Assessing Code Understandability: How Far Are We?": *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2017)*. 2017, 417–427. old. DOI: 10.1109/ASE.2017.8115654.

[6] Wendkûuni C. Ouédraogo és tsai. *Human-Aligned Code Readability Assessment with Large Language Models*. Preprint. 2025. arXiv: 2510.16579.

[7] Simone Scalabrino és tsai. „Improving Code Readability Models with Textual Features". *Proceedings of the 24th IEEE International Conference on Program Comprehension (ICPC 2016)*. 2016.

[8] B. Szalontai és tsai. „The Role of Code Readability in Large Language Model Code Summarization". *Proceedings of the 7th International Conference*

on Advances in Signal Processing and Artificial Intelligence (ASPAI 2025). Innsbruck, Austria, 2025, 148–154. old.